
EMERITUS
Learn. From the world's best.

Hands-On Clustering: From Theory to Practice

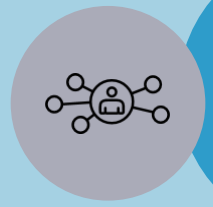
EMERITUS
Learn. From the world's best.

Clustering Concepts Recap

From the Async Session

K-Means:

- Partitions data into K clusters by minimising inertia (sum of squared distances to centroids)
- Sensitive to choice of K and feature scaling



Hierarchical:

- Builds a dendrogram of nested clusters
- No need to pre-set cluster number

Evaluating Clusters with Silhouette Score

Measure How Well Points Fit In Their Clusters

The Silhouette Score shows how well each data point fits within its cluster

Cohesion: Similarity within a cluster

Separation: Difference between clusters

For a point i:

Silhouette(i) $= \frac{b(i)-a(i)}{max(a(i),b(i))}$

Where:

a(i): Average distance to points in the same cluster

b(i): Lowest average distance to points in other clusters

Evaluating Clusters with Silhouette Score

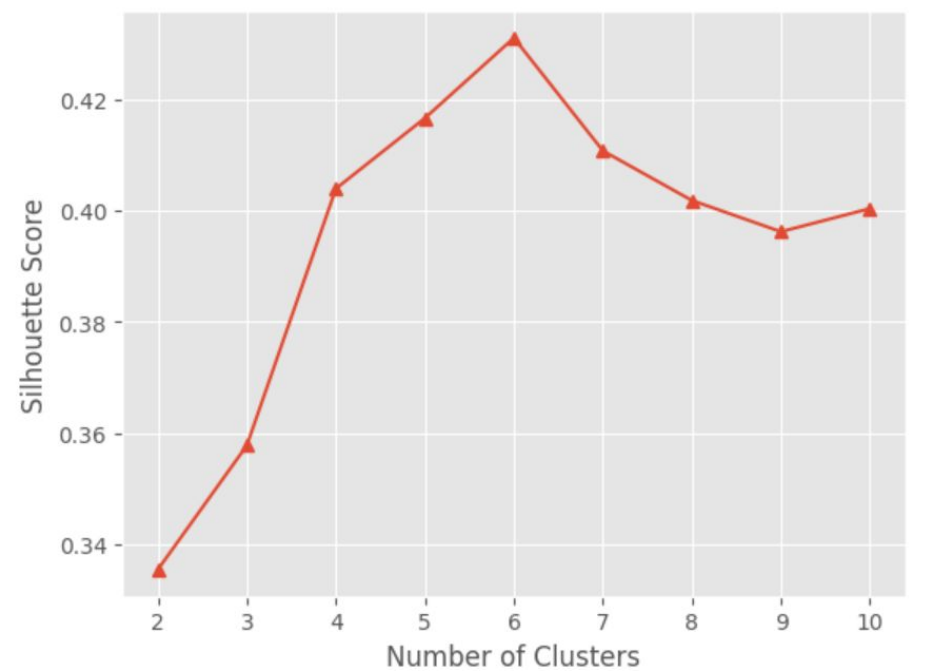
Measure How Well Points Fit In Their Clusters

- 1 = Optimal: Well-separated clusters
- 0 = Neutral: Overlapping clusters
- -1 = Poor: Misclassified points

```
from sklearn.metrics import silhouette_scorescore =
silhouette_score(Xs, labels)print(f"Silhouette Score:
{score:.3f}")
```

Visualising Silhouette Across K

Find The Best Number Of Clusters



Silhouette scores across K help identify the optimal number of clusters, complementing the Elbow Method

```
ks = range(2, 10)
scores = []
for k in ks:
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    labels = km.fit_predict(Xs)
    scores.append(silhouette_score(Xs, labels))
plt.plot(ks, scores, marker="o")
plt.xlabel("Number of clusters")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score vs K")
plt.show()
```

The highest silhouette score indicates the most natural K for clustering

Clustering Implementation

Environment & Data Setup

Required Libraries

```
# !pip install -U scikit-learn scipy matplotlib pandas numpy
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.datasets import load_iris, fetch_20newsgroups
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, MiniBatchKMeans
from sklearn.cluster import AgglomerativeClustering, DBSCAN
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.metrics import normalized_mutual_info_score
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
np.random.seed(42)  # For reproducibility
```

Environment & Data Setup

Datasets We'll Use

- **Iris**: Test algorithms on flower data
- **Mall Customers**: Customer segmentation
- **20 Newsgroups**: Text clustering
- **Sample Image**: Colour quantisation

Important Preprocessing Steps

- Standardise features for distance-based algorithms
- *Fix random_state* for reproducibility

K-Means Implementation

Applying Clustering On The Iris Dataset

K-Means: Popular clustering algorithm using k-means++ for centroids

```
iris = load_iris(as_frame=True)
X = iris.data.values
Xs = StandardScaler().fit_transform(X)
km = KMeans(n_clusters=3, init="k-means++",n_init=10, random_state=42)
labels = km.fit_predict(Xs)print("Inertia:", km.inertia_)
print("Silhouette:", silhouette_score(Xs, labels))
```

Inertia: Sum of squared distances to centroids; lower = tighter clusters
Silhouette: Compares similarity within clusters vs other clusters

Elbow & Silhouette Methods for Finding K

Implementation

```
ks = range(2, 10)
inertias, sils = [ ], [ ]
for k in ks:
    km = KMeans(n_clusters=k, n_init=10,random_state=42)
    km.fit(Xs)
    inertias.append(km.inertia_)
    sils.append(silhouette_score(Xs, km.labels_))
```

Elbow Method: Find the bend in the inertia curve where extra clusters add little benefit
Silhouette Method: Select K with the highest silhouette score for the most natural clusters

MiniBatchKMeans for Large Datasets

Faster Clustering With Mini-batches

MiniBatchKMeans updates centroids with mini-batches, giving faster clustering for large datasets with similar quality

```
mbkm = MiniBatchKMeans(n_clusters=3,  batch_size=64,
n_init=10,     random_state=42)
mb_labels = mbkm.fit_predict(Xs)
print("MiniBatch inertia:", mbkm.inertia_)
print("Silhouette:", silhouette_score(Xs, mb_labels))
```

MiniBatchKMeans for Large Datasets

Faster Clustering With Mini-batches

Sample Batch: Random subset of points

Update Centroids: Adjust using the batch

Iterate: Repeat until convergence

When to Use:

- Very large datasets
- Limited memory
- Need speed (10–50× faster)

Hierarchical Clustering

Tree-Based Clustering Without Preset K

- Hierarchical clustering creates a tree of clusters without needing a preset number
- **Linkage Types**:
  - **Ward**: Minimises variance (default)
  - **Complete**: Uses maximum distance
  - **Average**: Uses mean distance
- The dendrogram shows cluster hierarchy and can be cut at different heights to choose cluster numbers

Agglomerative Clustering (scikit-learn)

Easy-to-use hierarchical clustering

The scikit-learn implementation offers a more convenient interface for hierarchical clustering:

```
for link in ["ward", "complete", "average"]:
        model = AgglomerativeClustering(n_clusters=3,      inkage=link if link != "ward" else "ward",   metric='euclidean'    )
        lab = model.fit_predict(Xs)
        print(link, "silhouette:",
        silhouette_score(Xs, lab))
```

Advantages

- No need to set K in advance
- Captures hierarchy in data
- Offers multiple linkage options

Limitations

- Memory heavy: $O(n^2)$
- Slow runtime: $O(n^3)$
- Sensitive to noise and outliers

DBSCAN: Density-Based Clustering

Finds Clusters Of Any Shape And Detects Noise

DBSCAN finds clusters of any shape and detects noise automatically

```
from sklearn.neighbors import NearestNeighbors
nn = NearestNeighbors(n_neighbors=5)
nn.fit(Xs)dists, _ = nn.kneighbors(Xs)
plt.plot(np.sort(dists[:, -1]))
plt.title("k-distance plot (k=5)")
plt.show()
db = DBSCAN(eps=0.7, min_samples=5)
db_labels = db.fit_predict(Xs)
print("Noise points:", np.sum(db_labels == -1))
```

Key Parameters

eps: Maximum distance between two points to be considered neighbors

min_samples: Minimum neighbors required to form a core point

The k-distance plot finds the optimal *eps* at the elbow point

Cluster Evaluation

Metrics For Assessing Clustering Quality

Internal Metrics

Used when true labels are unknown

- **Inertia**: Sum of squared distance to centroids
- **Silhouette**: Cohesion vs separation
- **Davies-Bouldin**: Scatter vs separation ratio

External Metrics

Used when true labels are available

- **ARI**: Similarity between clusterings
- **NMI**: Information-based measure
- **Homogeneity / Completeness / V-measure**: Class assignment quality

Cluster Evaluation

Metrics For Assessing Clustering Quality

```
true_y = iris.target.values
print("ARI:", adjusted_rand_score(true_y, labels))
print("NMI:", normalized_mutual_info_score(true_y, labels))
```

Case Study

Metrics For Assessing Clustering Quality

Colour quantisation reduces image colours while preserving visual similarity, aiding compression and style

Process

- Reshape image into RGB pixels
- Use K-Means to select representative colours
- Replace each pixel with nearest centroid colour
- Reshape to original dimensions

```
from matplotlib.image import imread
img = imread('sample.jpg')[:, :, :3]
pixels = img.reshape(-1, 3)
km = KMeans(n_clusters=16).fit(pixels)
new_pixels = km.cluster_centers_[km.labels_]
new_pixels = new_pixels.reshape(img.shape)
```

Case Study

Metrics For Assessing Clustering Quality

This technique reduces millions of colours to about 16 while keeping the image appearance

Practical Tips and Assignment

Apply clustering methods and evaluate results

Tips

- Scale features for distance-based algorithms
- Use multiple initialisations for stability
- Try MiniBatchKMeans on large datasets
- Use DBSCAN for irregular shapes
- Apply dimensionality reduction for visualisation only

Practical Tips and Assignment

Apply clustering methods and evaluate results

Assignment

1. Pick a dataset from your domain
2. Apply K-Means and DBSCAN
3. Compare using silhouette and other metrics
4. If labels exist, calculate ARI and NMI
5. Visualise results and interpret meaning
6. Submit a short report with code and visuals

Remember: The best algorithm depends on your data and use case. Experiment and evaluate carefully

Demo

Q&A Session

- Open floor for questions and clarifications.